



# Circuitos Combinacionales

**Tema: Circuitos Lógicos**

**Módulo: Lógica Computacional y Algoritmos**

Prof. Lic. Tejerina Moran, Fernanda

# Circuitos lógicos

Los circuitos lógicos para sistemas digitales pueden ser

- **Combinacionales**
- **Secuenciales**

Un **circuito combinacional** consiste en compuertas lógicas cuyas salidas en cualquier momento están determinadas por la combinación actual de entradas. Un circuito combinacional realiza una operación que se puede especificar lógicamente con un conjunto de funciones booleanas.

Los **circuitos secuenciales** usan elementos de almacenamiento además de compuertas lógicas, y sus salidas son función de las entradas y del estado de los elementos de almacenamiento. Esto último, a su vez, es función de entradas anteriores. Por ello, las salidas de un circuito secuencial dependen no sólo de los valores actuales de las entradas, sino también de entradas anteriores, y el comportamiento del circuito se debe especificar con una sucesión temporal de entradas y estados internos.

# Circuitos Combinacionales

Son aquellas configuraciones de  $n$  entradas y  $m$  salidas, tal que su salida o salidas dependen exclusivamente del estado actual de sus variables de entrada, independientemente del factor tiempo.



# Circuitos Combinacionales

- **Circuitos Aritméticos**  
**Sumadores**  
**Restadores**  
**ALU**
- **De transferencia de datos**  
**Multiplexores**  
**Demultiplexores**  
**Comparadores**
- **De codificación**  
**Codificadores**  
**Decodificadores**  
**Conversores**

# Circuitos Combinacionales aritméticos, SUMA

Las computadoras digitales efectúan diversas tareas de procesamiento de información. Entre esas funciones están las operaciones aritméticas.

La operación aritmética más básica es la suma de dos dígitos binarios. Esta suma simple consiste en cuatro posibles operaciones elementales:  $0+0=0$ ,  $0+1=1$ ,  $1+0=1$  y  $1+1=10$ .

Las primeras tres operaciones producen una suma de un dígito, pero cuando ambos bits sumandos son 1, la suma binaria consta de dos dígitos. El bit más significativo de este resultado se denomina acarreo (carry, en inglés). Cuando ambos sumandos contienen más dígitos significativos, el acarreo obtenido de la suma de dos bits se suma al siguiente par más alto de bits significativos.

Un circuito combinacional que realiza la suma de dos bits se denomina semisumador; uno que realiza la suma de tres bits (dos bits significativos y un acarreo previo) es un sumador completo. Los nombres de los circuitos provienen del hecho de que es posible usar dos semisumadores para implementar un sumador completo.

# SUMADOR

## Sumadores

### Semisumador

| X | Y | C <sub>0</sub> | S |
|---|---|----------------|---|
| 0 | 0 | 0              | 0 |
| 0 | 1 | 0              | 1 |
| 1 | 0 | 0              | 1 |
| 1 | 1 | 1              | 0 |

### Sumador Completo

| X | Y | C <sub>-1</sub> | C <sub>0</sub> | S |
|---|---|-----------------|----------------|---|
| 0 | 0 | 0               | 0              | 0 |
| 0 | 0 | 1               | 0              | 1 |
| 0 | 1 | 0               | 0              | 1 |
| 0 | 1 | 1               | 1              | 0 |
| 1 | 0 | 0               | 0              | 1 |
| 1 | 0 | 1               | 1              | 0 |
| 1 | 1 | 0               | 1              | 0 |
| 1 | 1 | 1               | 1              | 1 |

# SUMADORES: Semisumador

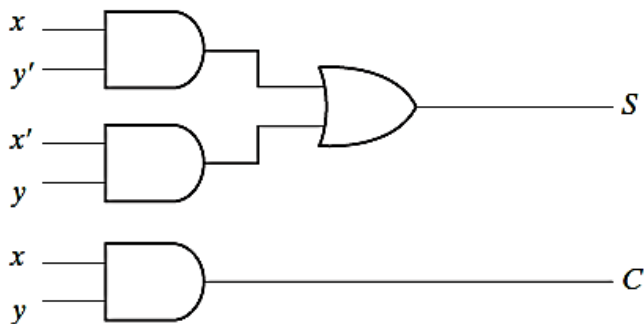
Semisumador

| $x$ | $y$ | $C$ | $S$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 0   |
| 0   | 1   | 0   | 1   |
| 1   | 0   | 0   | 1   |
| 1   | 1   | 1   | 0   |

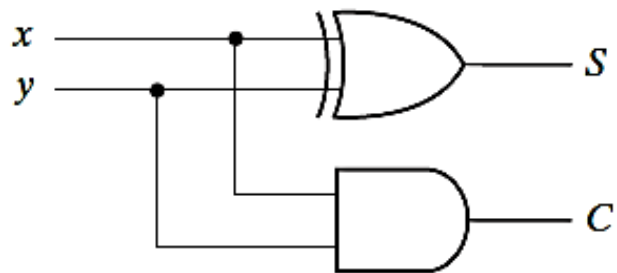
Las funciones booleanas simplificadas para las dos salidas se obtienen directamente de la tabla de verdad. Las expresiones simplificadas en suma de productos son:

$$S = x'y + xy' = x \oplus y$$

$$C = xy$$



a)  $S = xy' + x'y$   
 $C = xy$



b)  $S = x \oplus y$   
 $C = xy$

# SUMADORES: Sumador Completo

## Sumador Completo

| X | Y | Z | C | S |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 1 |

Tiene tres entradas y dos salidas. Dos de las variables de entrada, denotadas por  $x$  y  $y$ , representan los dos bits significativos que se sumarán. La tercera entrada, representa el acarreo de la posición significativa inmediata inferior.

Se requieren dos salidas porque la suma aritmética de tres dígitos binarios puede tener valores entre 0 y 3, y el 2 o el 3 binarios requieren dos dígitos.

La variable binaria  $S$  da el valor del bit menos significativo de la suma. La variable binaria  $C$  da el acarreo de salida.

Del mapa de karnaugh surgen las siguientes expresiones simplificadas:

$$S = x'y'z + x'yz' + xy'z' + xyz$$

$$C = xy + xz + yz$$

$$\begin{aligned} S &= z \oplus (x \oplus y) \\ &= z'(xy' + x'y) + z(xy' + x'y)' \\ &= z'(xy' + x'y) + z(xy + x'y') \\ &= xy'z' + x'yz' + xyz + x'y'z \end{aligned}$$

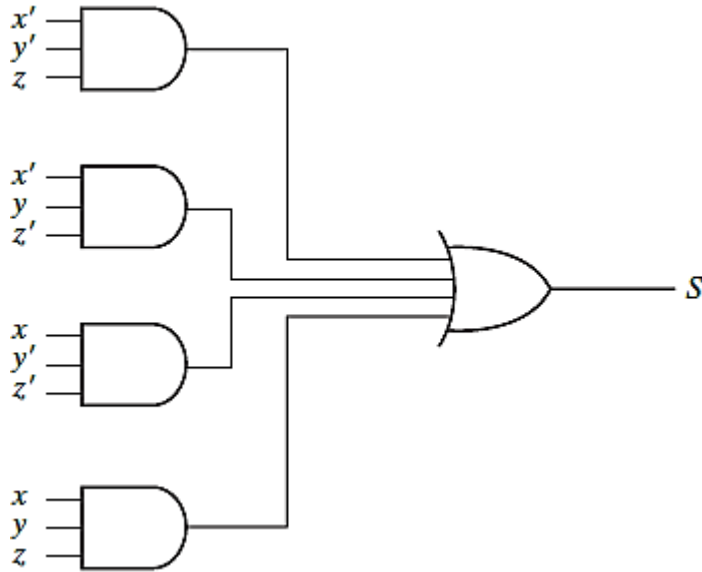
$$C = z(xy' + x'y) + xy = xy'z + x'yz + xy$$



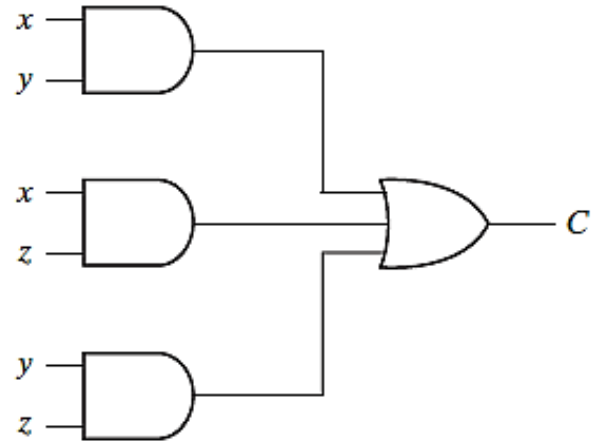
# SUMADORES: Sumador Completo

Sumador Completo

$$S = x'y'z + x'yz' + xy'z' + xyz$$



$$C = xy + xz + yz$$

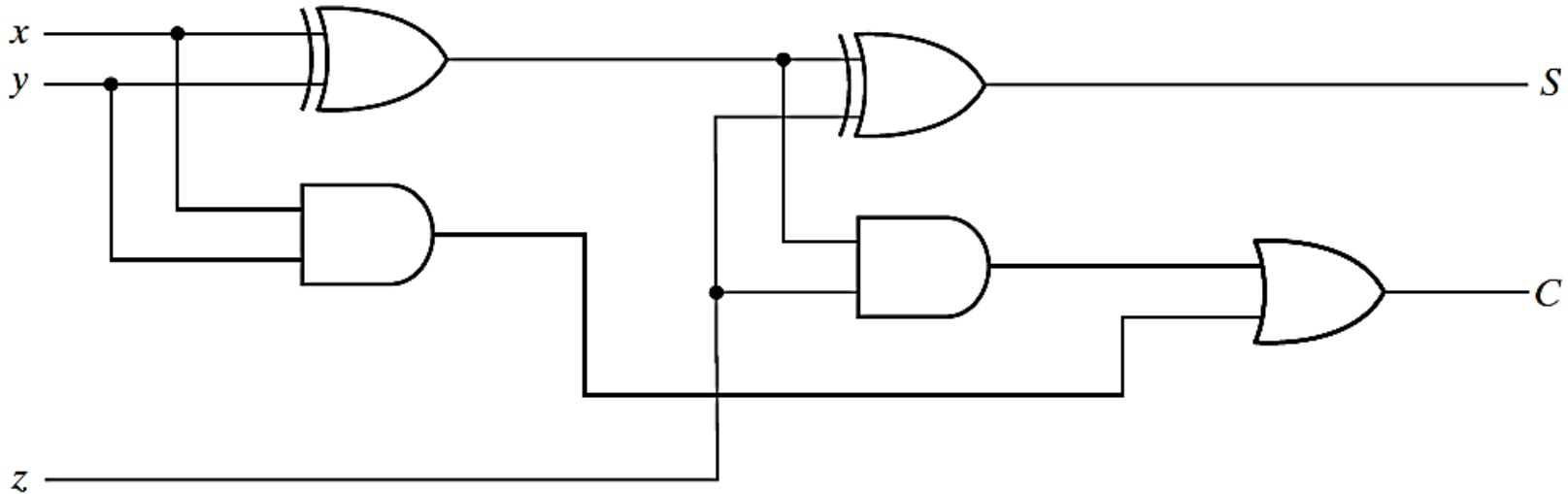


# SUMADORES: Sumador Completo

Sumador Completo

$$S = z \oplus (x \oplus y)$$

$$C = z(xy' + x'y) + xy = xy'z + x'yz + xy$$

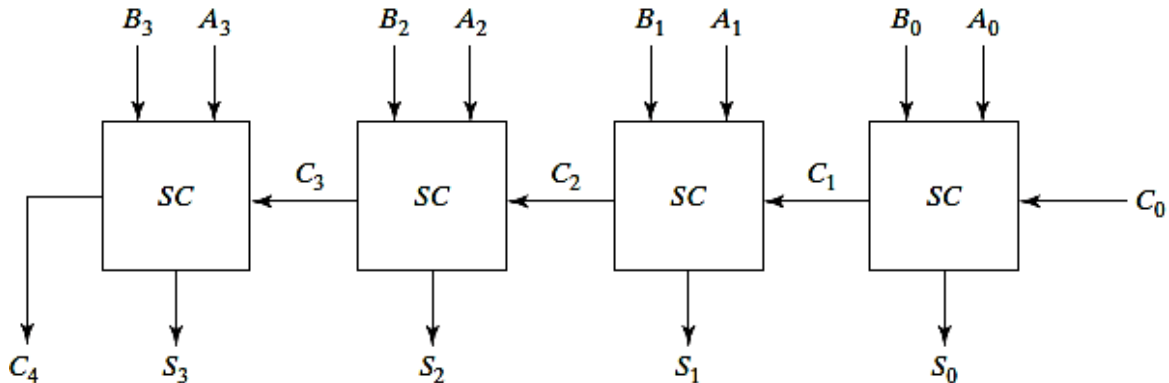


# SUMADORES: Sumador Binario

Un sumador binario es un circuito digital que produce la suma aritmética de dos números binarios. Es posible construirlo con sumadores completos dispuestos en cascada, conectando el acarreo de salida de cada sumador completo al acarreo de entrada del siguiente sumador completo de la cadena.

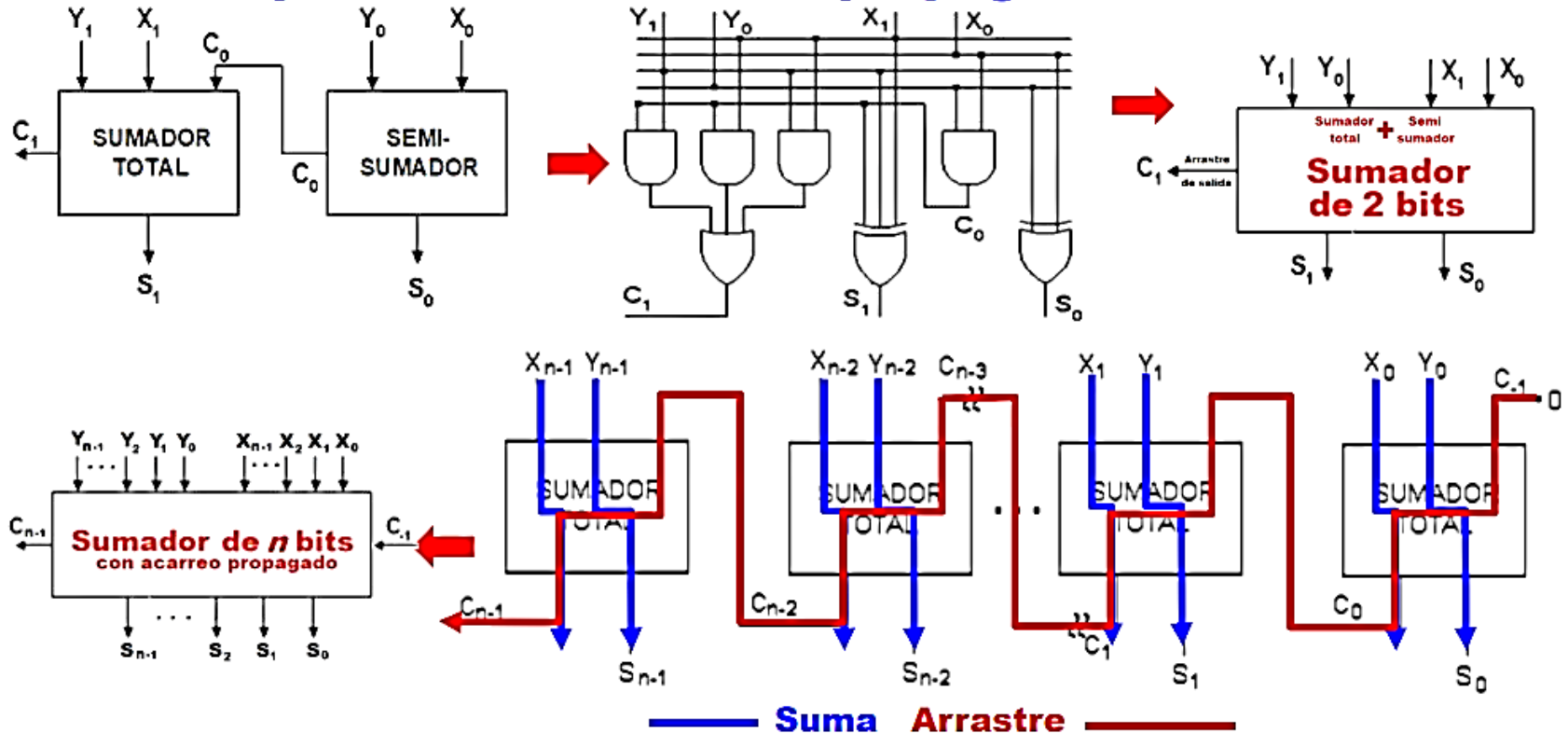
Consideremos dos números binarios  $A=1011$  y  $B=0011$ . Su suma  $S=1110$  se forma con el sumador de cuatro bits así

| Subíndice $i$ :    | 3 | 2 | 1 | 0 |           |
|--------------------|---|---|---|---|-----------|
| Acarreo de entrada | 0 | 1 | 1 | 0 | $C_i$     |
| Sumando            | 1 | 0 | 1 | 1 | $A_i$     |
| Sumando            | 0 | 0 | 1 | 1 | $B_i$     |
| Suma               | 1 | 1 | 1 | 0 | $S_i$     |
| Acarreo de salida  | 0 | 0 | 1 | 1 | $C_{i+1}$ |



# SUMADORES: Sumador Binario

## Sumador paralelo con arrastre propagado

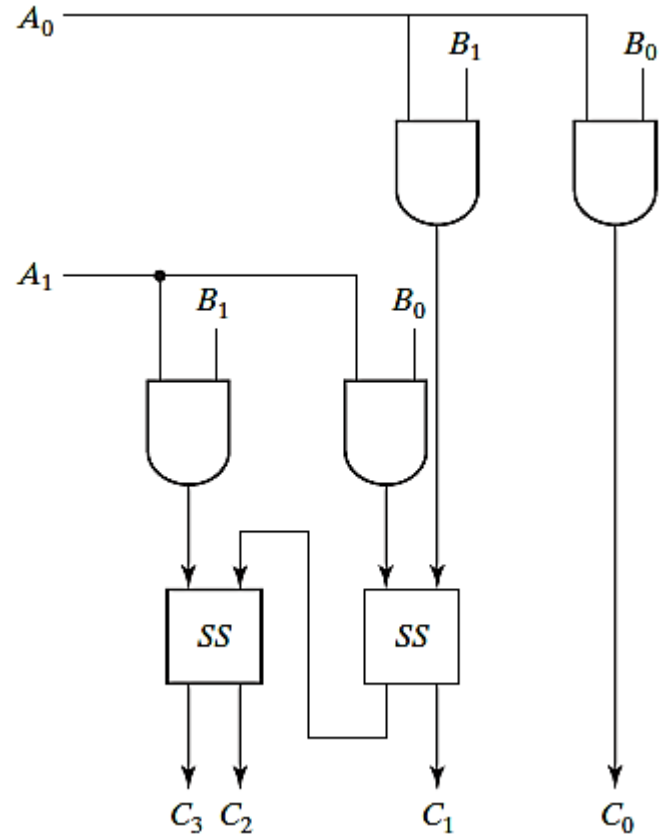


Dormido, pg. 188

# MULTIPLICADOR BINARIO

Consideremos la multiplicación de dos números de dos bits, como se muestra en la figura. Los bits del multiplicando son  $B_1$  y  $B_0$ , los bits del multiplicador son  $A_1$  y  $A_0$ , y el producto es  $C_3C_2C_1C_0$ .

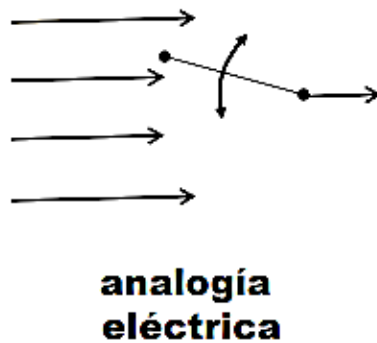
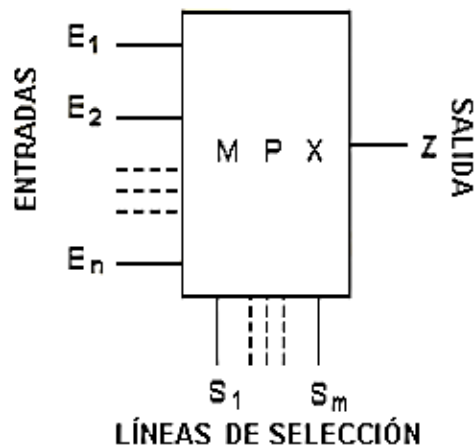
|       |          |          |       |
|-------|----------|----------|-------|
|       | $B_1$    | $B_0$    |       |
|       | $A_1$    | $A_0$    |       |
|       | <hr/>    |          |       |
|       | $A_0B_1$ | $A_0B_0$ |       |
|       | $A_1B_1$ | $A_1B_0$ |       |
| <hr/> |          |          |       |
| $C_3$ | $C_2$    | $C_1$    | $C_0$ |



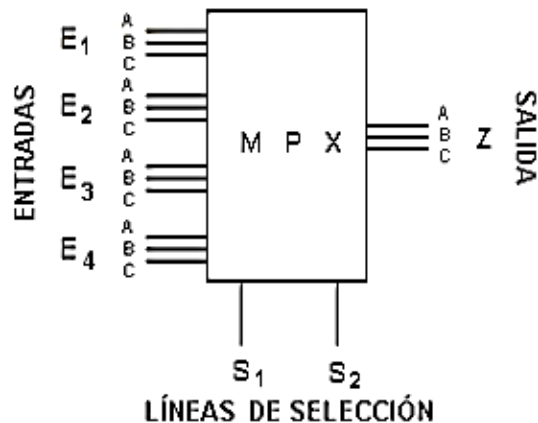
# MULTIPLEXOR

Un multiplexor es un circuito combinacional que selecciona información binaria de una de muchas líneas de entrada y la envía a una sola línea de salida. La selección de una línea de entrada dada se controla con un conjunto de líneas de selección. Normalmente, hay  $2^n$  líneas de entrada y  $n$  líneas de selección cuyas combinaciones de bits determinan cuál entrada se selecciona.

**MPX  $n \times 1$**   
 **$n$  entradas/canales**  
 **$\times 1$  bit/canal**



**MPX  $4 \times 3$**   
**4 entradas/canales**  
 **$\times 3$  bits/canal**

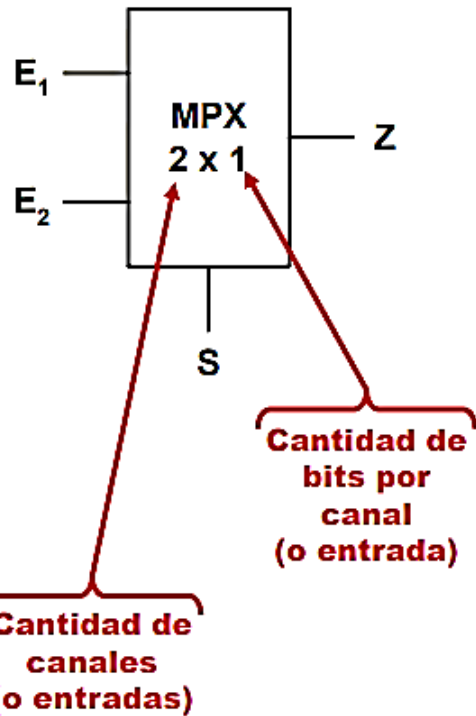
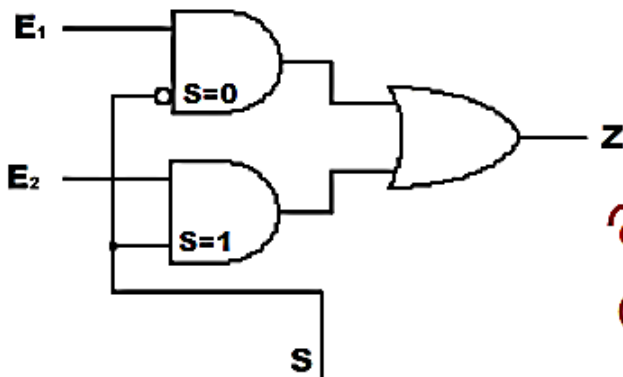


# MULTIPLEXOR

## Ejemplo: Multiplexor de 2x1

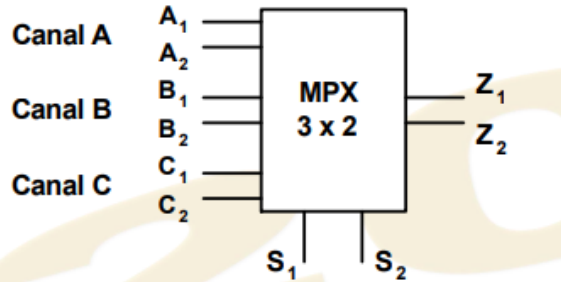
$$Z = E_1 \cdot \bar{S} + E_2 \cdot S$$

| S | E <sub>1</sub> | E <sub>2</sub> | Z |
|---|----------------|----------------|---|
| 0 | 0              | X              | 0 |
| 0 | 1              | X              | 1 |
| 1 | X              | 0              | 0 |
| 1 | X              | 1              | 1 |



# MULTIPLEXOR

## Ejemplo: Multiplexor de 3x2

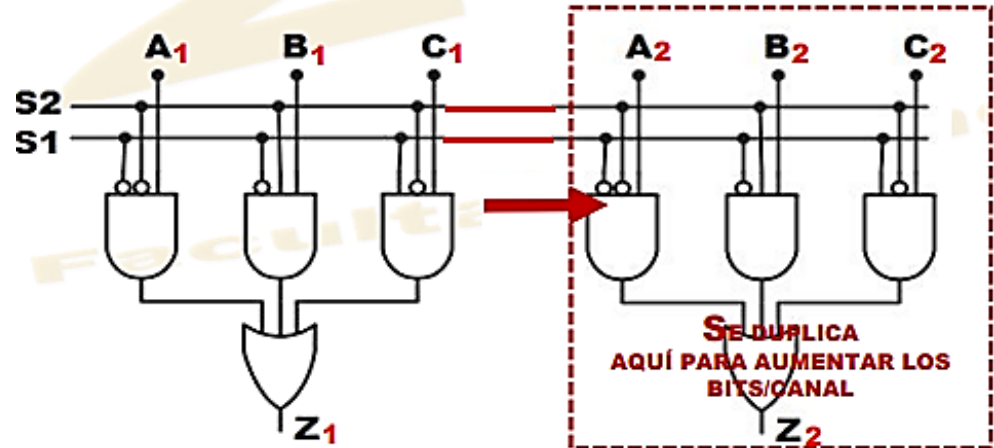


$$\text{MPX 4X1} \quad Z = \overline{S1}.\overline{S2}.A + \overline{S1}.S2.B + S1.\overline{S2}.C + S1.S2.D$$

$$\text{MPX 3X1} \quad Z = \overline{S1}.\overline{S2}.A + \overline{S1}.S2.B + S1.\overline{S2}.C$$

Conviene siempre iniciar con un diseño MPX  $2^N \times 1$ , luego reproducir la estructura hasta conseguir los bits/canal requeridos.

| Selección |    | Datos |   |   |   | Salida |
|-----------|----|-------|---|---|---|--------|
| S1        | S2 | A     | B | C | D | Z      |
| 0         | 0  | 0     | X | X | X | 0      |
| 0         | 0  | 1     | X | X | X | 1      |
| 0         | 1  | X     | 0 | X | X | 0      |
| 0         | 1  | X     | 1 | X | X | 1      |
| 1         | 0  | X     | X | 0 | X | 0      |
| 1         | 0  | X     | X | 1 | X | 1      |
| 1         | 1  | X     | X | X | 0 | 0      |
| 1         | 1  | X     | X | X | 1 | 1      |





# MULTIPLEXOR

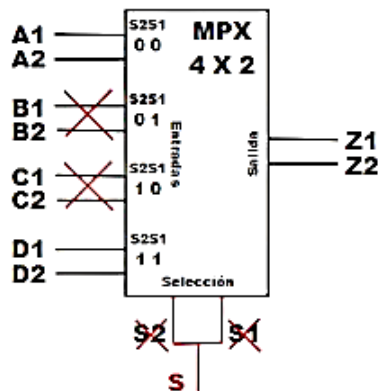
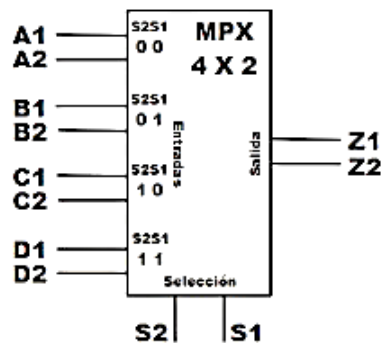
## Multiplexores (MPX) – Asociación

Combinar o asociar multiplexores permite crear estructuras más complejas, utilizando bloques menores, con las que se puede multiplexar datos para (relativamente) cualquier número de canales y de bits por canal. Los criterios a seguir son simples:

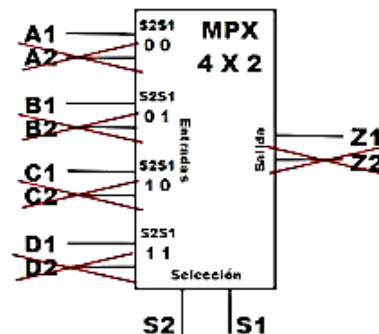
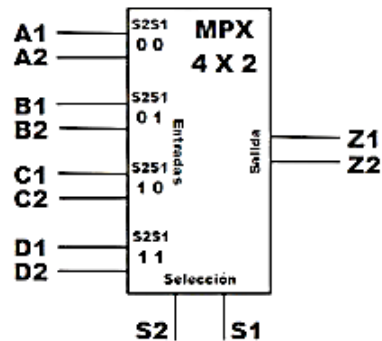
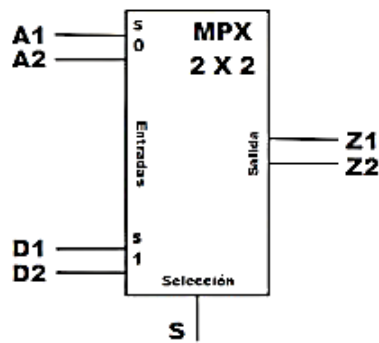
- Al aumentar la cantidad de canales, generalmente debe aumentar la cantidad de líneas de selección, tal que  $N^{\circ} \text{ canales} \leq 2^{(N^{\circ} \text{ líneas selección})}$   
significa: por lo menos una palabra de código de selección para cada canal.
- Al aumentar la cantidad de bits por canal, no aumentan las líneas de selección. La nueva estructura debe conectar sus líneas de selección 'en bus' con las ya existentes.
- Para el diseño de estructuras de  $N \times M$  ( $N$ =canales;  $M$ =bits), es buena práctica iniciar con un bloque de  $N \times 1$ , multiplicar este formato y conectarlos, hasta alcanzar los  $M$  bits por canal.
- Multiplexores de  $N \times M$  se pueden reducir en cantidad de canales, omitiendo los que no se necesitan. Idem para los bits/canal.

# MULTIPLEXOR

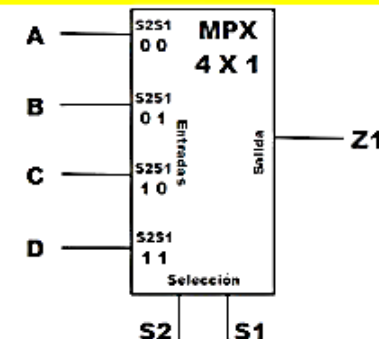
## Ejemplo: Reducción de multiplexores



### Reducción de canales



### Reducción de bits/canal

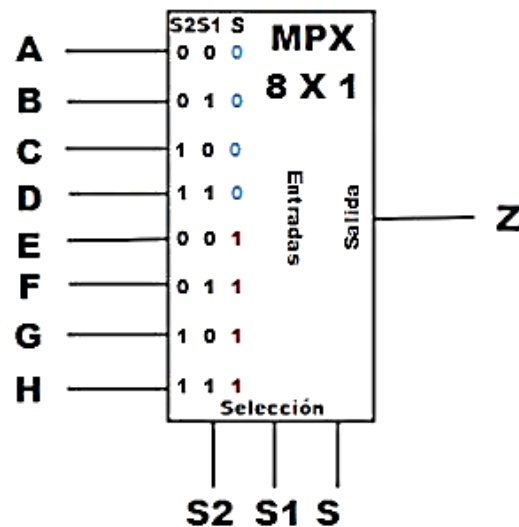
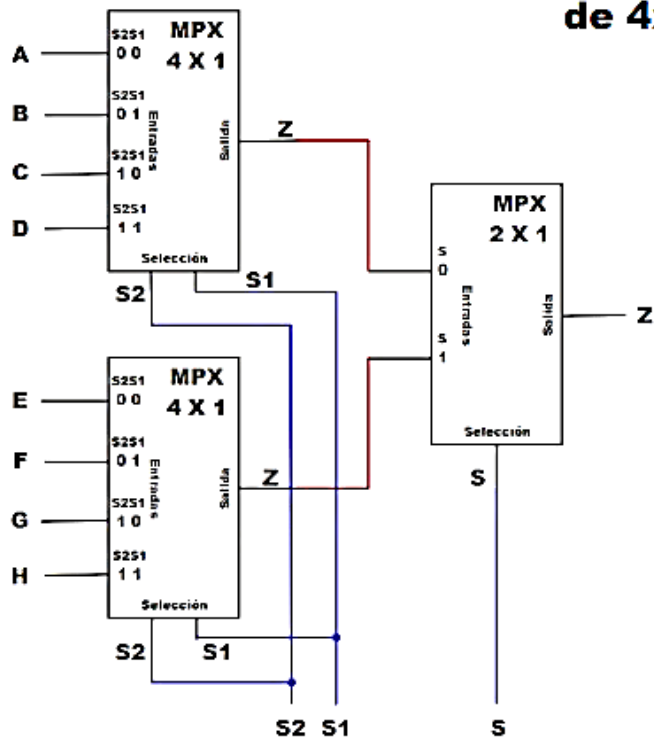


# MULTIPLEXOR

## Multiplexor (MPX) - Asociación

**Ejemplo:** Aumentar la cantidad de canales de 4x1 a 8x1

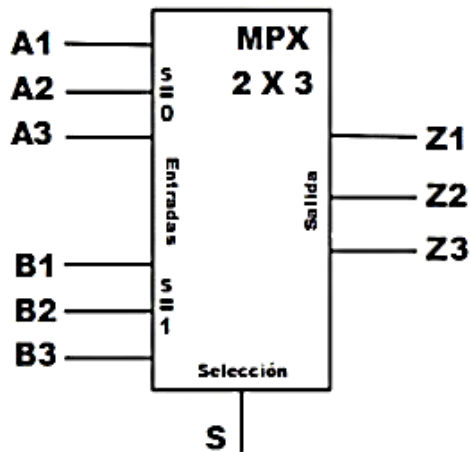
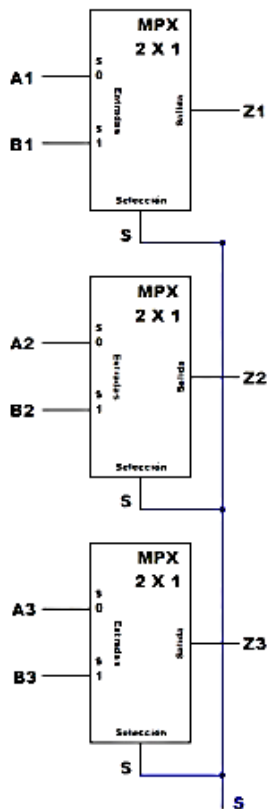
Para aumentar una línea de selección se agrega una nueva instancia de decisión (MPX 2x1) que seleccione las salidas del primer grupo.



# MULTIPLEXOR

## Multiplexores (MPX) - Asociación

**Ejemplo:** Aumentar la cantidad de bits por canal, de 2x1 a 2x3.

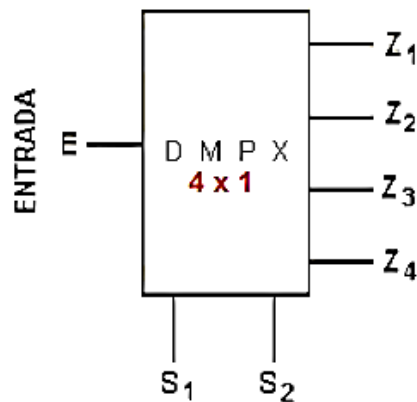


**Al aumentar la cantidad de bits por canal, no se modifica la cantidad de líneas de selección. La estrategia es conectar en bus, unidades con la misma cantidad de canales.**

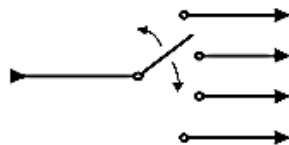
# DEMULTIPLEXORES

Es un circuito combinacional de una entrada y varias salidas, que puede “transferir” (reconstruir) la entrada a una de las salidas, mediante una codificación predefinida.

**4 salidas/canales  
× 1 bit/canal**



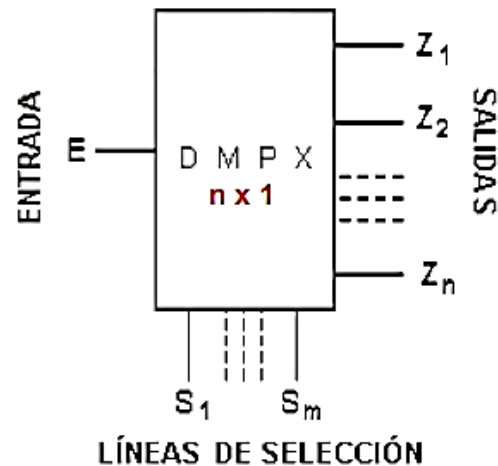
ENTRADA



SALIDAS

**analogía  
eléctrica**

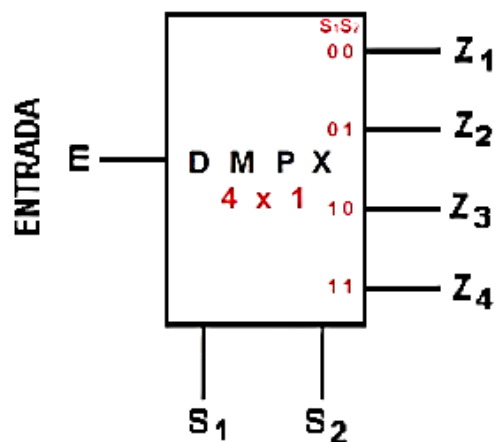
**n salidas/canales  
× 1 bit/canal**



# DEMULTIPLEXORES

## Demultiplexores (DMPX)

## Ejemplo: Demultiplexor de 4x1 – Código binario natural



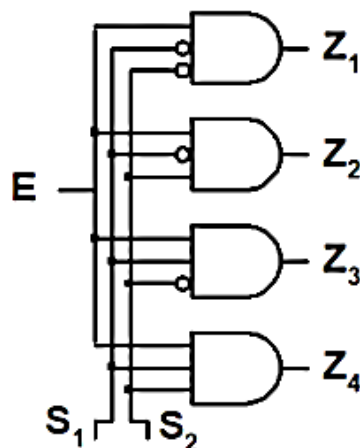
| S <sub>1</sub> | S <sub>2</sub> | E | Z <sub>1</sub> | Z <sub>2</sub> | Z <sub>3</sub> | Z <sub>4</sub> |
|----------------|----------------|---|----------------|----------------|----------------|----------------|
| 0              | 0              | 0 | 0              | X              | X              | X              |
| 0              | 0              | 1 | 1              | X              | X              | X              |
| 0              | 1              | 0 | X              | 0              | X              | X              |
| 0              | 1              | 1 | X              | 1              | X              | X              |
| 1              | 0              | 0 | X              | X              | 0              | X              |
| 1              | 0              | 1 | X              | X              | 1              | X              |
| 1              | 1              | 0 | X              | X              | X              | 0              |
| 1              | 1              | 1 | X              | X              | X              | 1              |

$$Z_1 = \overline{S_1} \cdot \overline{S_2} \cdot E$$

$$Z_2 = \overline{S_1} \cdot S_2 \cdot E$$

$$Z_3 = S_1 \cdot \overline{S_2} \cdot E$$

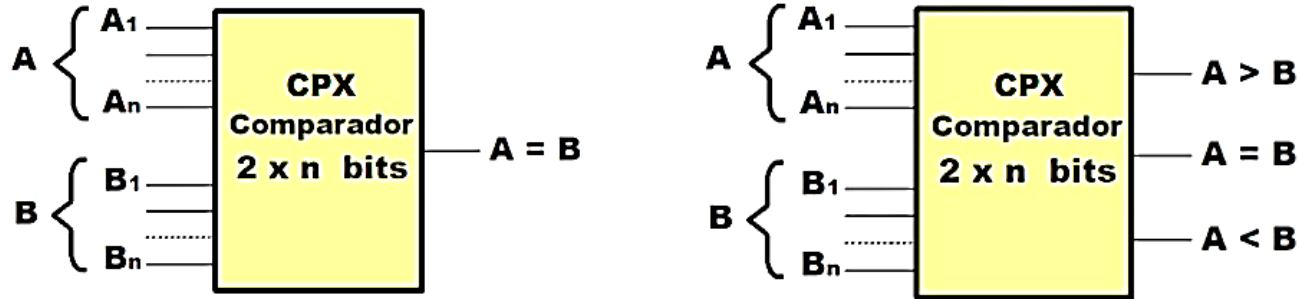
$$Z_4 = S_1 \cdot S_2 \cdot E$$



La síntesis **directa** es simple. Tantas AND como salidas. A cada AND ingresa la entrada **E** y las líneas de selección codificadas.

# COMPARADORES

La comparación de dos números es una operación que determina si un número es mayor que, menor que o igual a otro número. Un comparador de magnitudes es un circuito combinacional que compara dos números, A y B, y determina sus magnitudes relativas. El resultado de la comparación se especifica con tres variables binarias que indican si  $A > B$ ,  $A = B$  o  $A < B$ .

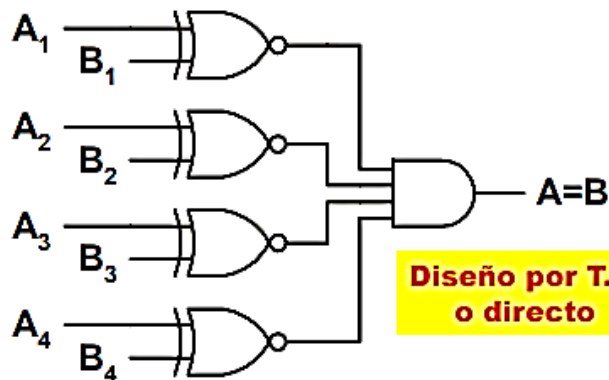
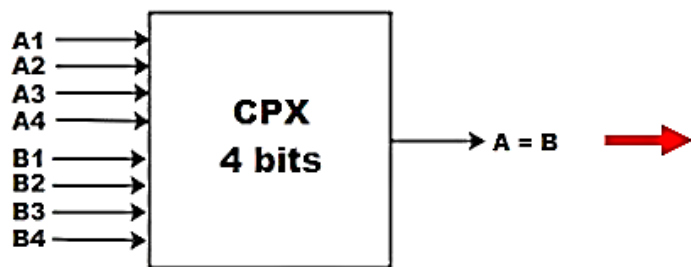


**Bloques genéricos**

# COMPARADORES

## Comparadores (CPX)

**Ejemplo:** Diseño de un comparador de 4 bits a igualdad.



**Diseño por T.V.  
o directo**

| Dec              | A1 | A2 | A3 | A4 | B1 | B2 | B3 | B4 | Z1 |
|------------------|----|----|----|----|----|----|----|----|----|
| 0 <sub>10</sub>  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  |
| 1 <sub>10</sub>  | 0  | 0  | 0  | 1  | 0  | 0  | 0  | 1  | 1  |
| 2 <sub>10</sub>  | 0  | 0  | 1  | 0  | 0  | 0  | 1  | 0  | 1  |
| 3 <sub>10</sub>  | 0  | 0  | 1  | 1  | 0  | 0  | 1  | 1  | 1  |
| 4 <sub>10</sub>  | 0  | 1  | 0  | 0  | 0  | 1  | 0  | 0  | 1  |
| 5 <sub>10</sub>  | 0  | 1  | 0  | 1  | 0  | 1  | 0  | 1  | 1  |
| 6 <sub>10</sub>  | 0  | 1  | 1  | 0  | 0  | 1  | 1  | 0  | 1  |
| 7 <sub>10</sub>  | 0  | 1  | 1  | 1  | 0  | 1  | 1  | 1  | 1  |
| 8 <sub>10</sub>  | 1  | 0  | 0  | 0  | 1  | 0  | 0  | 0  | 1  |
| 9 <sub>10</sub>  | 1  | 0  | 0  | 1  | 1  | 0  | 0  | 1  | 1  |
| 10 <sub>10</sub> | 1  | 0  | 1  | 0  | 1  | 0  | 1  | 0  | 1  |
| 11 <sub>10</sub> | 1  | 0  | 1  | 1  | 1  | 0  | 1  | 1  | 1  |
| 12 <sub>10</sub> | 1  | 1  | 0  | 0  | 1  | 1  | 0  | 0  | 1  |
| 13 <sub>10</sub> | 1  | 1  | 0  | 1  | 1  | 1  | 0  | 1  | 1  |
| 14 <sub>10</sub> | 1  | 1  | 1  | 0  | 1  | 1  | 1  | 0  | 1  |
| 15 <sub>10</sub> | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 1  |

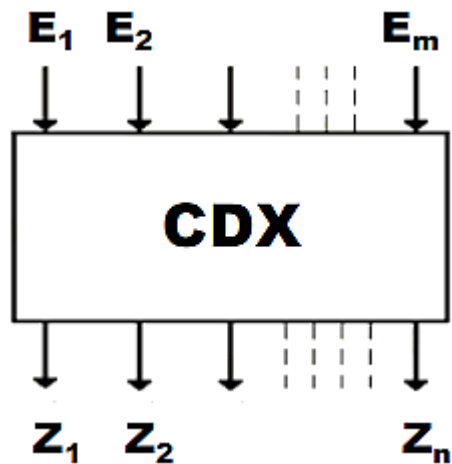
Las restantes combinaciones toman valor 0



# CODIFICADORES

Son circuitos combinacionales de  $m$  entradas y  $n$  salidas, que al recibir por las entradas una palabra binaria no codificada (ej. código posicional), produce a la salida la palabra equivalente codificada.

El proceso de diseño consiste básicamente en diseñar un circuito combinacional que asocie cada palabra de entrada con la correspondiente palabra de salida codificada



### Código posicional decimal (1 de 10)

[illegible]

# CODIFICADORES

## Codificadores (CDX)

**Ejemplo: Codificador sin prioridad**  
de cuaternario  $(4,1)_{ss}$  a binario natural  $(2,2)_{ss}$

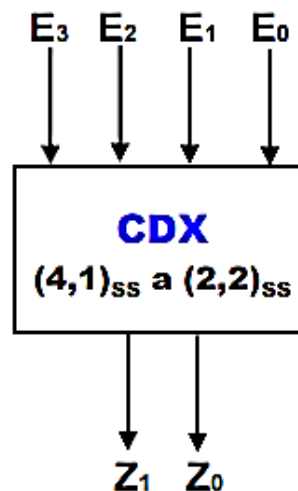


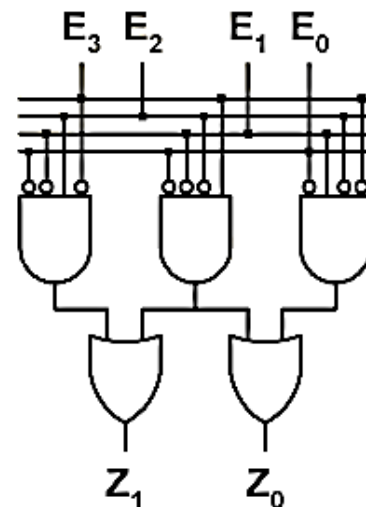
Diagrama de bloque

Disño  
por T.V.

| Posicional              |       |       |       | Binario |       |
|-------------------------|-------|-------|-------|---------|-------|
| $E_3$                   | $E_2$ | $E_1$ | $E_0$ | $Z_1$   | $Z_0$ |
| 0                       | 0     | 0     | 1     | 0       | 0     |
| 0                       | 0     | 1     | 0     | 0       | 1     |
| 0                       | 1     | 0     | 0     | 1       | 0     |
| 1                       | 0     | 0     | 0     | 1       | 1     |
| Restantes combinaciones |       |       |       | X       | X     |

Tabla de verdad reducida  
**SIN PRIORIDAD**

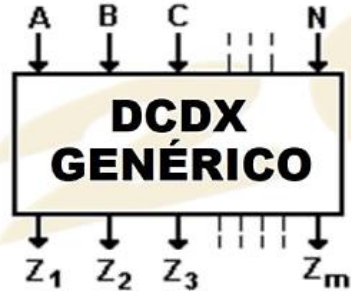
Implementación  
canónica



Circuito lógico

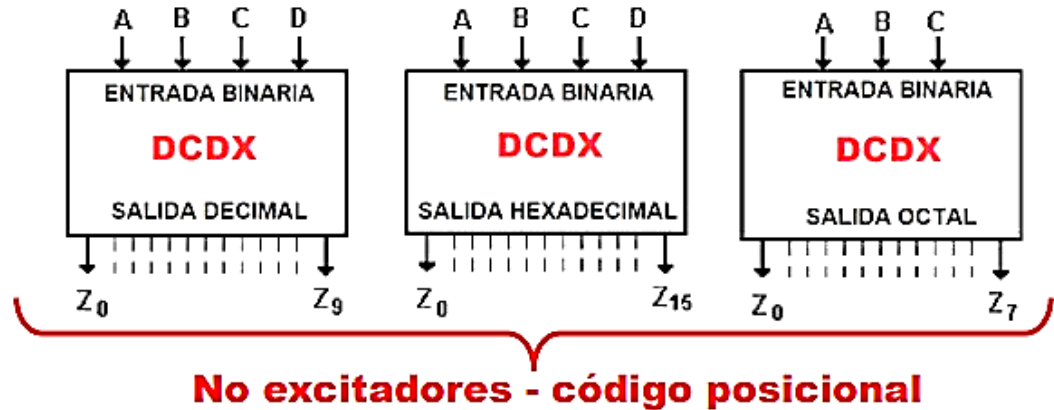
# DECODIFICADORES

Son circuitos combinacionales de  $N$  líneas de entradas y  $m$  líneas de salida, que al recibir una secuencia binaria en un código preestablecido, produce a la salida un valor equivalente sin codificar (código posicional).



**Decodificadores**

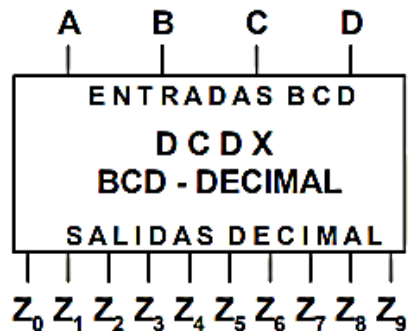
- No excitadores - salida binaria posicional
- Excitadores
  - 7 segmentos
  - 14 segmentos
  - matriz de puntos
  - otros visualizadores



# DECODIFICADORES

## Decodificadores (DCDX)

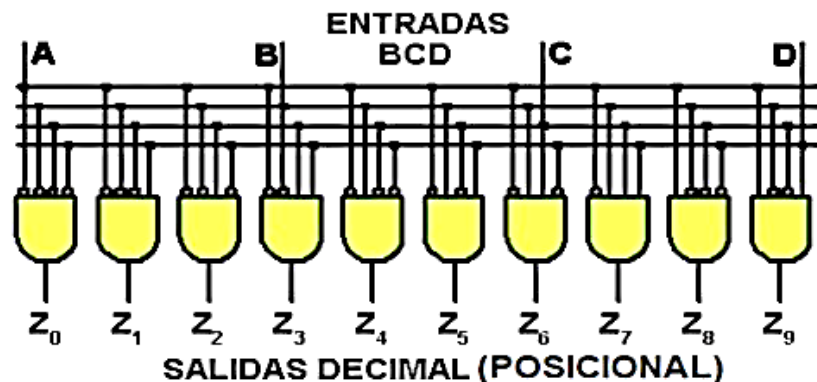
**Ejemplo: Decodificador BCD natural a decimal no excitador.**



| # | A B C D | Z <sub>0</sub> | Z <sub>1</sub> | Z <sub>2</sub> | Z <sub>3</sub> | Z <sub>4</sub> | Z <sub>5</sub> | Z <sub>6</sub> | Z <sub>7</sub> | Z <sub>8</sub> | Z <sub>9</sub> |
|---|---------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|
| 0 | 0 0 0 0 | 1              | 0              | 0              | 0              | 0              | 0              | 0              | 0              | 0              | 0              |
| 1 | 0 0 0 1 | 0              | 1              | 0              | 0              | 0              | 0              | 0              | 0              | 0              | 0              |
| 2 | 0 0 1 0 | 0              | 0              | 1              | 0              | 0              | 0              | 0              | 0              | 0              | 0              |
| 3 | 0 0 1 1 | 0              | 0              | 0              | 1              | 0              | 0              | 0              | 0              | 0              | 0              |
| 4 | 0 1 0 0 | 0              | 0              | 0              | 0              | 1              | 0              | 0              | 0              | 0              | 0              |
| 5 | 0 1 0 1 | 0              | 0              | 0              | 0              | 0              | 1              | 0              | 0              | 0              | 0              |
| 6 | 0 1 1 0 | 0              | 0              | 0              | 0              | 0              | 0              | 1              | 0              | 0              | 0              |
| 7 | 0 1 1 1 | 0              | 0              | 0              | 0              | 0              | 0              | 0              | 1              | 0              | 0              |
| 8 | 1 0 0 0 | 0              | 0              | 0              | 0              | 0              | 0              | 0              | 0              | 1              | 0              |
| 9 | 1 0 0 1 | 0              | 0              | 0              | 0              | 0              | 0              | 0              | 0              | 0              | 1              |

*Combinaciones restantes no utilizadas*

Nótese que en este caso el diseño **directo** es simple: una compuerta AND por salida de código y cada una recibe a la entrada las líneas de datos codificadas con negadores.

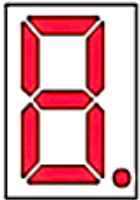


# DECODIFICADORES

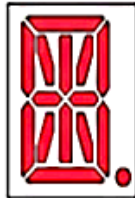
## Decodificadores (DCDX) excitadores

Los decodificadores identificados como excitadores, en vez de dar salida en un código posicional, se diseñan para generar un código especial que sea capaz de activar algún dispositivo visualizador, interpretable por un usuario. Los dispositivos visualizadores (displays) más comunes se muestran abajo, siendo el de 7 segmentos el más usual.

**7  
segmentos**



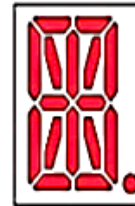
**14  
segmentos**



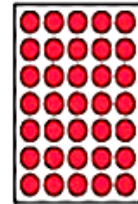
**tubo nixie**



**16 segmentos  
alfanumérico**

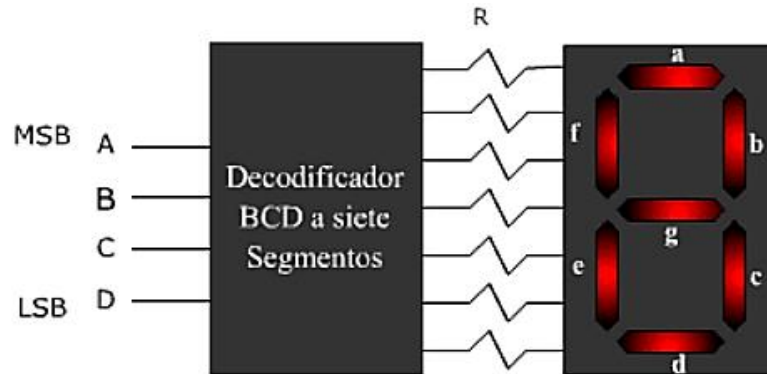


**matriz de puntos  
alfanumérico**



# DECODIFICADORES

Ejemplo: Decodificadores BCD a 7 segmentos Es un circuito combinacional que permite un código BCD en sus entradas y en sus salidas activa un display de 7 segmentos para indicar un dígito decimal. El display está formado por un conjunto de 7 leds conectados en un punto común en su salida, bien en ánodo común ó ánodo común.

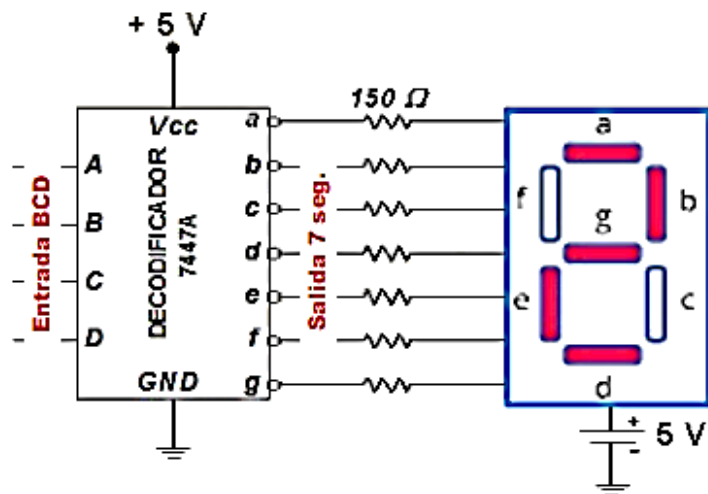


# DECODIFICADORES

Cada segmento del display se identifica con una letra (de **a** hasta **g**). La estrategia para construir la TV es marcar con '1' aquellos segmentos que deban formar el dígito.

| Entrada BCD |   |   |   | Salida 7 segmentos |   |   |   |   |   |   |   |
|-------------|---|---|---|--------------------|---|---|---|---|---|---|---|
| dec         | A | B | C | D                  | a | b | c | d | e | f | g |
| 0           | 0 | 0 | 0 | 0                  | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
| 1           |   |   | 0 | 1                  | 0 | 0 | 0 | 0 |   |   |   |
| 2           |   |   | 1 | 0                  | 1 | 1 | 0 | 1 |   |   |   |
| 3           |   |   | 1 | 1                  | 1 | 1 | 0 | 0 |   |   |   |
| 4           | 0 | 1 | 0 | 0                  | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| 5           |   |   | 0 | 1                  | 1 | 0 | 1 | 1 |   |   |   |
| 6           |   |   | 1 | 0                  | 1 | 1 | 1 | 1 |   |   |   |
| 7           |   |   | 1 | 1                  | 1 | 0 | 0 | 0 |   |   |   |
| 8           | 1 | 0 | 0 | 0                  | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 9           |   |   | 0 | 1                  | 1 | 1 | 1 | 0 | 1 | 1 |   |

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 0 |
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 0 |



Un procedimiento similar se puede seguir para configurar el código de salida en los otros displays, sólo tener en cuenta la cantidad de segmentos y el diseño de los caracteres.

# DECODIFICADORES

Los decodificadores se emplean fundamentalmente para seleccionar los diferentes puertos de E/S ( entrada/salida) y así la computadora pueda comunicarse con los diferentes dispositivos externos ( periféricos). Estos decodificadores son conocidos como decodificador de direcciones de puertos.

Direccionar una localidad de memoria, conversión de datos binarios,...